

Web Semântica

Proposta / Implementação / Avaliação Preliminar

Pedro Saito

Universidade Federal do Rio de Janeiro

2026-06-24



1. Proposta

Construir uma engine federada mínima para consultas SPARQL, mensurável pelo benchmark FedShop.

FedX

Implementação base

Seleção de fontes sem metadados com ASK
exclusive groups, bound joins, joins reorder

pyfedx

Protótipo em Python

Parsing, ASK, cache, junções locais

Sem *bound joins* ou *exclusive groups*

Semagrow / Costfed / ANAPSID

Otimização com metadados e assíncrono

Planejamento por custo e fontes *join-aware*

Execução adaptativa e tolerância a lentidão.

go-fed

Implementação final em Go

Seleção de fontes com ASK ou *summaries*
bound joins, exclusive groups, custos e
artefatos FedShop

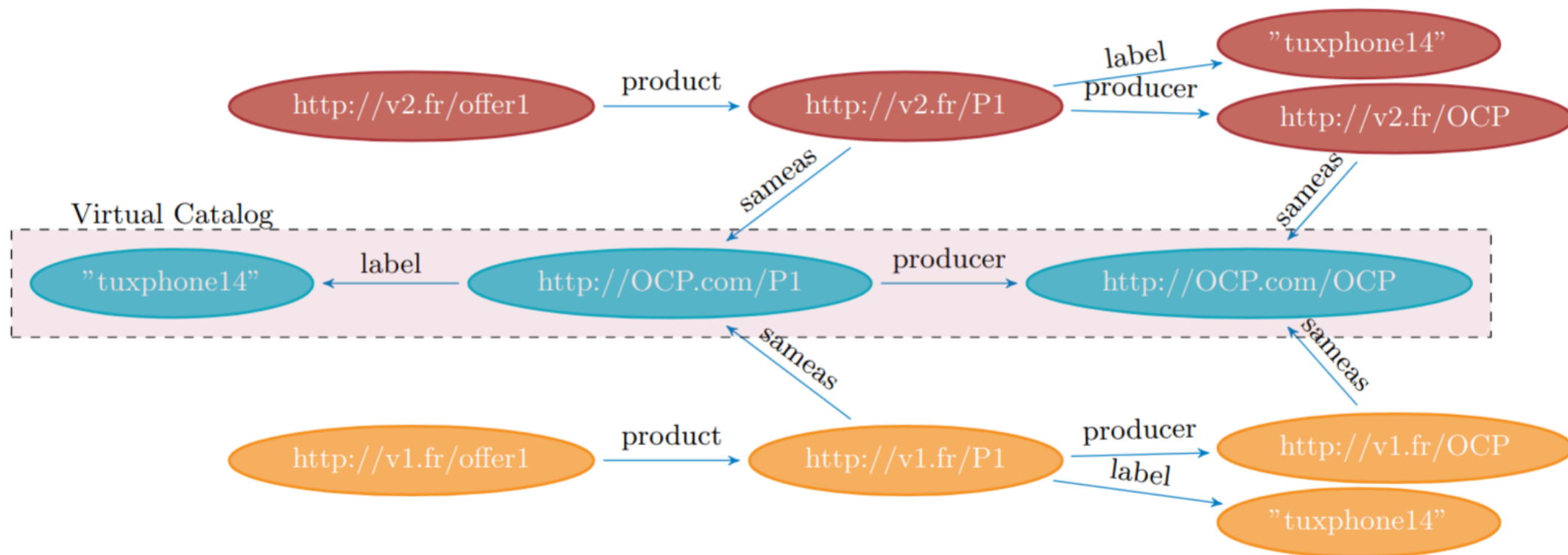


Figura 1: *Catálogo virtual e replicação de produtos entre vendedores*

FedShop

Benchmark de escalabilidade de engines SPARQL federadas.

O objetivo é isolar o impacto do tamanho da federação e da estratégia da engine.

Eixo	Variações consideradas
Consultas	12 <i>templates</i> de consultas
Instâncias	10 valores concretos por template
Escala	10 <i>batches</i> : 20, 40, ..., 200 membros
Fontes	vendedores + sites de avaliação
Tentativas	Repetições controladas
Engines	GoFed, FedX, CostFed, protótipos..

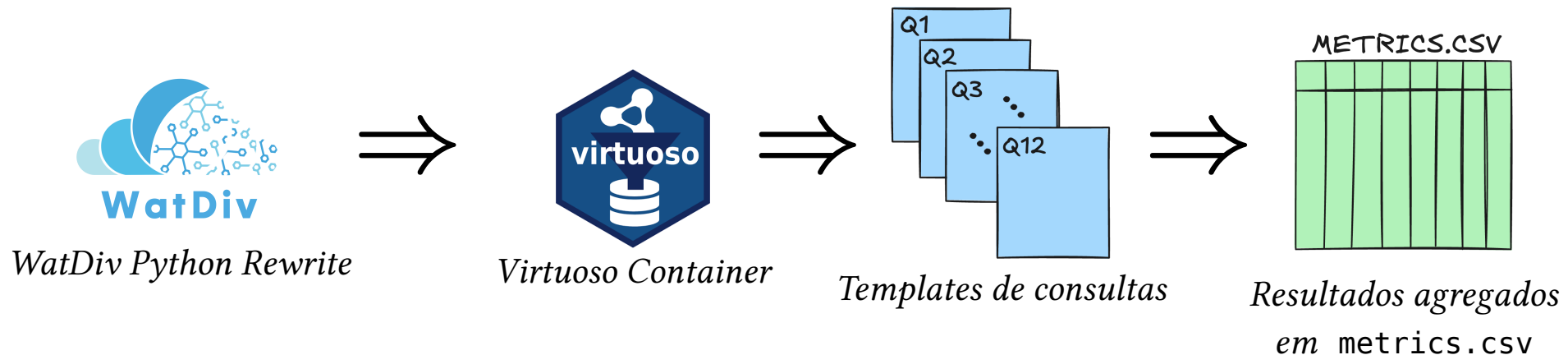
Cada execução fixa:

Engine × **Consulta** × **Instância** × **Batch** × **Tentativa**

2. Implementação

2.1 Pipeline Reprodutível

1. Geração de *n-quads* com WatDiv – Gerador de dados RDF sintéticos usando YAML.
2. Ingestão no *Virtuoso* – Plataforma escalável para banco de dados, integração e LD.
3. Instanciação das consultas – 12 *templates* SPARQL formatados com valores do *dataset*.
4. Execução das *engines* – adapters padronizam chamadas, *timeouts* e coleta de resultados.
5. Agregação das métricas – Tempo de execução, n° de requisições



Validar decisões de arquitetura antes da implementação em Go, com o objetivo de reduzir incertezas.

Mantidos

Parsing SPARQL

Source Selection

Execução federada

Métricas

Artefatos FedShop

Componente	Simplificação
Planejamento	Sem otimização por custo
Execução	Sequencial
Metadados	Apenas ASK + cache
Paralelismo	Não implementado
Escalabilidade	Foco em validação

2.3 pyfedx: Diagrama

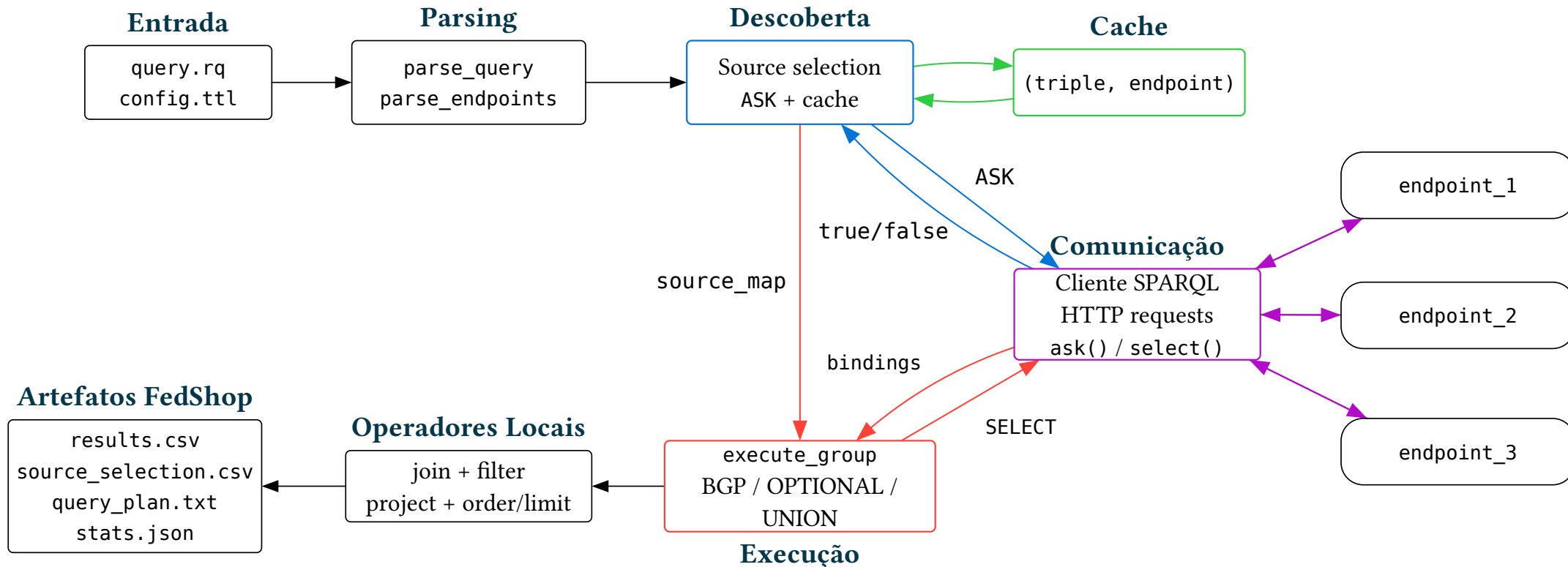


Figura 6: Fluxo interno do pyfedx: seleção de fontes por ASK, execução HTTP e composição local dos resultados.

[link para diagrama go-fed](#)

3. Avaliação

Consulta de referência em que a seleção de fontes é conhecida antecipadamente e inserida por meio de cláusulas SERVICE.

Consulta SPARQL Original

```
SELECT ?product ?label WHERE {  
  ?product rdfs:label ?label ;  
           rdf:type bsbm:ProductType ;  
           bsbm:productFeature bsbm:FeatureA ;  
           bsbm:productFeature bsbm:FeatureB ;  
           bsbm:price ?price .  
  FILTER (?price > 744)  
}  
LIMIT 10
```

sparql

Consulta SPARQL com SERVICE

```
SELECT ?product ?label WHERE {  
  SERVICE <http://vendor1.fr/sparql> {  
    ?product rdfs:label ?label ;  
             rdf:type bsbm:ProductType ;  
             bsbm:productFeature bsbm:FeatureA ;  
             bsbm:productFeature bsbm:FeatureB ;  
             bsbm:price ?price .  
    FILTER (?price > 744)  
  }  
}  
LIMIT 10
```

sparql

Listagem 1: *Consulta original* → *Consulta RSA (fontes pré-atribuídas)*

Dimensão	Métricas	Alvo
Corretude	status nb_results	Execução e resultados
Tempo	exec_time planning_time source_selection_time	Custos das etapas
Rede	ask http_req data_transfer	Custos de comunicação
Seleção de fontes	nb_distinct_sources relevant_sources_selectivity	Variedade de fontes selecionadas
Qualidade da seleção	tpwss, avg_rwss, min_rwss, max_rwss	Qtd e importância de fontes

Query	go-fed			FedX			PyFedX		
	✓	#	Δt	✓	#	Δt	✓	#	Δt
q01	100%	10	7.3s	0%	TO	—	0%	TO	—
q02	0%	TO	—	0%	TO	—	0%	TO	—
q04	50%	10	13s	0%	TO	—	0%	TO	—
q05	100%	5	9.3s	0%	TO	—	0%	TO	—
q06	100%	13.5	0.32s	75%	9	40s	50%	9	79s
q07	100%	21	0.26s	100%	0	101s	100%	21	9.3s
q08	100%	1	0.13s	100%	1	1.56s	100%	1	1.16s
q09	100%	1	0.02s	100%	1	0.66s	100%	1	0.08s
q10	100%	1	0.08s	100%	0	25.08s	100%	1	0.58s
q11	100%	10	0.05s	100%	10	1.09s	100%	10	0.15s
q12	100%	21	0.12s	100%	21	2.18s	100%	21	12.37s

Tabela 1: **FedShop benchmark summary**. Suc. = success, Res. = average results, TO = timeout.

Ambiente atual

Linguagem	Python 3.12
CLI	<code>click + uv</code>
SPARQL	<code>rdflib /</code> <code>SPARQLWrapper</code>
Geração de dados	<i>WatDiv</i>
Banco de dados	<i>Virtuoso</i>
Engine final	go-fed em Golang

Sequência de entrega

1. Completar o núcleo da engine em Go.
2. Escrever *adapters* para *engines* restantes
5. Avaliar ablação e benchmark completo.
6. Gerar métricas agregadas com todas *engines*.

Referências

- (1) Fernandes, L. C.; Santos Ribeiro, L. dos; Castro, M. V. B. de; Silva Pacheco, L. A. da; Oliveira Sandes, E. F. de. JurisTCU: A Brazilian Portuguese Information Retrieval Dataset with Query Relevance Judgments. *arXiv preprint* **2025**.
- (2) Lv, Y.; Zhai, C. X. When Documents Are Very Long, BM25 Fails!. Em *Proceedings of the 34th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '11)*; 2011; pp 1103–1104. <https://doi.org/10.1145/2009916.2010070>.
- (3) Li, X.; Lipp, J.; Shakir, A.; Huang, R.; Li, J. BMX: Entropy-weighted Similarity and Semantic-enhanced Lexical Search. *arXiv preprint* **2024**.
- (4) Bruch, S.; Gai, S.; Ingber, A. An Analysis of Fusion Functions for Hybrid Retrieval. *arXiv preprint* **2022**.
- (5) Silva Júnior, e. a. Datasets for Portuguese Legal Semantic Textual Similarity. *Language Resources and Evaluation* **2023**.
- (6) —. Análise da Eficácia de Fine-Tuning de Embeddings no Contexto Jurídico Brasileiro. — **202AD**.
- (7) —. Building a Relevance Feedback Corpus for Legal Information Retrieval in the Real-Case Scenario of the Brazilian Chamber of Deputies. Em —; 202AD.
- (8) Lin, J.; Ma, X.; Lin, S.-C.; Yang, J.-H.; Pradeep, R.; Nogueira, R. Pyserini: A Python Toolkit for Reproducible Information Retrieval Research with Sparse and Dense Representations. Em *Proceedings of the 44th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2021)*; 2021; pp 2356–2362.
- (9) Bassani, E. ranx: A Blazing-Fast Python Library for Ranking Evaluation and Comparison. Em *ECIR (2)*; Lecture Notes in Computer Science; Springer, 2022; Vol. 13186, pp 259–264. https://doi.org/10.1007/978-3-030-99739-7_30.